

1.2 Historical Trends in Deep Learning

This section provides an overview of how deep learning has evolved over time:

- **Changing Names and Popularity:** Deep learning has undergone multiple name changes and shifts in popularity since its origins in the 1940s.
- **Data and Scale:** The effectiveness of deep learning has increased due to the availability of more data and improved computing power.
- **Growing Model Complexity:** Deep learning models have become larger and more sophisticated, leading to improvements in various applications.
- **Real-World Impact:** Deep learning has revolutionized fields such as image recognition (e.g., ImageNet Challenge) and speech recognition.

1.3 The Many Names and Evolution of Neural Networks

This section discusses the three historical waves of artificial neural network research:

1. **Cybernetics (1940s-1960s):** Early models like the McCulloch-Pitts neuron and perceptron were introduced.
2. **Connectionism (1980s-1990s):** Introduction of backpropagation and multi-layer networks.
3. **Deep Learning (2006-present):** Advances in hardware (GPUs), algorithms (e.g., convolutional networks), and large datasets have enabled modern deep learning success.

1.4 Key Trends Driving Deep Learning

- **Hardware Advancements:** GPUs and specialized hardware like TPUs have accelerated deep learning progress.
- **Data Abundance:** Large-scale datasets like ImageNet have facilitated breakthroughs.
- **Algorithmic Improvements:** Techniques such as better optimization methods and regularization have refined deep learning performance.

Historical Trends in Deep Learning

Deep learning has evolved from simple **single-layer perceptrons** to **multi-layer architectures** capable of achieving human-like intelligence in various domains. With ongoing **research in unsupervised learning, self-supervised learning, and multimodal AI**, deep learning is expected to continue advancing rapidly.

Deep learning has evolved significantly over the decades, influenced by theoretical advancements, computing power, and access to large-scale data. The history of deep learning can be categorized into three major waves: **Cybernetics (1940s-1960s), Connectionism (1980s-1990s), and the Modern Deep Learning Era (2006-Present)**. Below is a detailed breakdown of these historical trends.

1. Early Developments (1940s-1960s) – The Cybernetics Era

Key Developments:

- The concept of **Artificial Neural Networks (ANNs)** was introduced, inspired by biological neurons.
- **McCulloch-Pitts Neuron (1943)** was the first mathematical model of a neural network. It was a **binary threshold model** that could perform basic logical operations.
- **Hebbian Learning Rule (1949):** Donald Hebb proposed that "neurons that fire together, wire together," suggesting that learning happens through synaptic strength adjustments.

Limitations & Decline:

- The **Perceptron**, introduced by **Frank Rosenblatt (1958)**, was a simple single-layer neural network capable of classification.
 - However, **Minsky and Papert (1969)** showed that perceptrons could not solve non-linearly separable problems (e.g., XOR function).
 - Due to these limitations, neural network research lost traction, leading to the first **AI Winter** (a period of reduced funding and interest in neural networks).
-

2. Revival of Neural Networks (1980s-1990s) – The Connectionism Era

Key Developments:

- The **backpropagation algorithm** was introduced in **1986** by **Rumelhart, Hinton, and Williams**, making it possible to train multi-layer neural networks.
- **Hinton & colleagues** demonstrated that neural networks could learn useful representations through **hidden layers**.
- **Convolutional Neural Networks (CNNs)** were introduced by **Yann LeCun** in the late 1980s for **image recognition**.
- **Recurrent Neural Networks (RNNs)** were developed to handle **sequential data**.

Limitations & Decline:

- Neural networks still faced computational challenges, as training deep networks required massive computational power that was unavailable at the time.
 - **Support Vector Machines (SVMs)** and **Kernel Methods** outperformed neural networks in many applications, leading to a second decline in interest during the late 1990s and early 2000s.
-

3. The Deep Learning Revolution (2006-Present)

Key Developments:

1. **Breakthrough in Unsupervised Learning (2006)**
 - **Geoffrey Hinton and collaborators** introduced **Deep Belief Networks (DBNs)**, using **greedy layer-wise pretraining** to train deep networks more effectively.
 - This marked the beginning of **modern deep learning**.
2. **Advancements in Hardware (2010s)**
 - **GPUs** (Graphics Processing Units) significantly accelerated neural network training.
 - Custom AI hardware like **Google's TPUs (Tensor Processing Units)** made deep learning more efficient.
3. **Big Data and Large-Scale Training**
 - The availability of **large labeled datasets** (e.g., **ImageNet** for image recognition, **LibriSpeech** for speech processing) enabled training of **large-scale neural networks**.
4. **Breakthroughs in Neural Network Architectures**
 - **Convolutional Neural Networks (CNNs)**: Achieved **state-of-the-art performance in image recognition** (e.g., AlexNet in 2012, ResNet in 2015).
 - **Recurrent Neural Networks (RNNs) & LSTMs**: Became widely used in **speech recognition and natural language processing (NLP)**.
 - **Transformers (2017)**: Revolutionized NLP with models like **BERT and GPT**, leading to today's generative AI boom.

Modern Applications of Deep Learning:

- **Computer Vision** (Self-driving cars, Medical Imaging)
- **Natural Language Processing** (Chatbots, Machine Translation)
- **Speech Recognition** (Virtual Assistants like Alexa, Google Assistant)
- **Generative AI** (Text-to-Image models, ChatGPT)

Computer Vision

Deep learning has significantly improved **image and video analysis**:

- **Image Classification & Object Detection**
 - Used in self-driving cars, medical diagnostics, and surveillance.
 - **YOLO (You Only Look Once)** and **Faster R-CNN** are popular object detection models.
- **Medical Imaging**
 - CNNs are used for diagnosing diseases (e.g., cancer detection from X-rays, MRIs).
- **Facial Recognition**
 - Used in security, authentication (Face ID), and law enforcement.

Natural Language Processing (NLP)

Transformers have led to **human-like language understanding**:

- **Machine Translation** (Google Translate improved with Transformers).
- **Text Generation** (ChatGPT, Bard, Claude).
- **Speech-to-Text & Virtual Assistants** (Siri, Alexa, Google Assistant).
- **Sentiment Analysis** (Used in finance, marketing, customer feedback).

Speech Recognition

- **End-to-End Deep Learning Models**
 - Traditional speech recognition required **hand-engineered features**.
 - Modern deep learning models directly learn from raw audio.
- **Applications:**
 - **Google Voice Search, Alexa, Siri, Cortana**
 - **Automatic transcription services** (Otter.ai, Rev)
 - **Real-time language translation**

Generative AI

Deep learning can now **generate human-like content**:

- **Text Generation:** GPT-3, ChatGPT, LLaMA models.
- **Image Generation:** DALL·E, MidJourney, Stable Diffusion.
- **Music & Video Generation:** AI tools like Suno and Runway ML.
- **Code Generation:** GitHub Copilot, AlphaCode.

Key Factors Driving Deep Learning's Success

1. **Increased Data Availability** – More data enables better generalization.
 2. **Better Computing Power** – GPUs, TPUs, and cloud computing accelerate model training.
 3. **Algorithmic Innovations** – Regularization techniques (e.g., Dropout), Optimizers (e.g., Adam), and new architectures (e.g., Transformers) improve learning efficiency.
-

Challenges & Future Directions in Deep Learning

Despite its success, deep learning faces several challenges:

Explainability & Interpretability

- **Black Box Problem:**
 - Deep neural networks **lack transparency**—it's difficult to explain why a model made a certain decision.
 - This is critical in **healthcare, finance, and law** where AI decisions impact people's lives.
- **Research Solutions:**
 - **SHAP, LIME:** Explainable AI techniques that interpret model decisions.
 - **Attention Visualization:** In NLP, attention mechanisms help understand what the model is focusing on.

Ethical Concerns & Bias

- **Bias in AI Models**
 - AI models inherit biases from training data. Example:
 - Facial recognition systems have shown **racial and gender bias**.
 - Chatbots like **ChatGPT can generate biased responses**.
- **Efforts to Reduce Bias**
 - **Fairness-aware AI:** Researchers work on de-biasing datasets and models.
 - **Regulations & AI Ethics Policies:** Governments are introducing AI regulations to ensure fair use.

Data Efficiency & Learning Without Labels

- **Deep learning requires huge labeled datasets.**
- **Self-supervised learning (SSL) & Unsupervised Learning**
 - Models like **SimCLR, BERT, and CLIP** learn from **unlabeled data**, reducing dependence on human-labeled datasets.

Energy Consumption & Sustainability

- Training large deep learning models requires massive computational resources.
- **GPT-3 alone consumed thousands of GPUs, leading to high energy costs.**
- **Solutions:**
 - **Efficient AI models** (e.g., DistilBERT, MobileNet).
 - **Quantum computing for AI** (still in early research).

Future of Deep Learning

- **Multimodal AI:** Models that can handle text, images, audio, and video together (e.g., GPT-4V, Gemini).
 - **Neurosymbolic AI:** Combining neural networks with symbolic reasoning for better interpretability.
 - **Continual Learning:** AI systems that **learn and adapt over time** without retraining.
-

UNIT-III

①

Deep feed forward Networks

* Deep feed fwd n/w's are also called as feed forward neural networks (or) multilayer perceptrons (MLP's)

→ The goal of a feed forward n/w is to approximate the

fn: f^*

i.e for a classifier, $y = f^*(x)$ maps i/p x to category y .

→ A feed forward n/w defines a mapping $y = f(x; \theta)$
It learns value of θ that ~~are~~ result in best function approximation.

→ These models are called feed forward because info. flows in forward direction.
(i.e from x , through intermediate computations used to define f , & finally to o/p y)

→ These are called n/w's because they are represented by composing together many different functions.

for ex: If we have 3 fns $f(1)$, $f(2)$, $f(3)$ connected -
we write it as $f(x) = f(3)(f(2)(f(1)(x)))$

here $f(1)$ is called first layer
 $f(2)$ " " second layer & so-on

→ The overall length of the chain, gives depth of the model.

So the name deep learning.

→ final layer is called output layer.

→ The training examples specify directly what the output layer must do at each point x . It must produce a value close to y .

→ The behaviour of the other layers is not directly specified by training data.

→ So these are called hidden layers

* → One way to understand feed forward n/ws is to begin with linear models.

* → To overcome the limitations of linear models, we can apply non linear transformations.

→ Feed forward n/ws have introduced the concept of hidden layers & this requires us to choose Activation fns that will be used to compute the hidden layer values.

* Example : Learning XOR

So, let's begin with an example of a fully connected feed forward n/w, i.e. learning the XOR function.

XOR fn:	x_1	x_2	y
	0	0	0
	0	1	1
	1	0	1
	1	1	0

It provides the target fn: $y = f^*(x)$ that we want to learn.

→ our model provides a fn: $y = f(x; \theta)$

∴ learning alg. will adapt the parameters θ to make f as similar as f^*

→ As this is a regression problem we use Mean Squared error loss fn: (MSE).

* evaluate this on our whole training set

MSE loss fn: is

$$J(\theta) = \frac{1}{4} \sum_{x \in X} (f^*(x) - f(x; \theta))^2 \quad \text{--- (1)}$$

[0, 0]
[0, 1]
[1, 0]
[1, 1]

→ function of our model is $f(x; \theta)$

with θ consisting of w & b
↳ wt ↳ bias

* our model is defined as

$$f(x; w, b) = x^T w + b \quad \text{--- (2)}$$

below is the fig. with explanation after applying transformation.

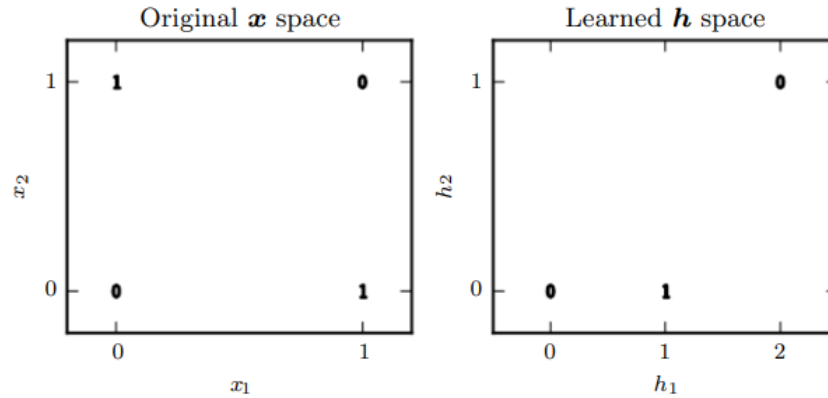


Figure 6.1: Solving the XOR problem by learning a representation. The bold numbers printed on the plot indicate the value that the learned function must output at each point. *(Left)* A linear model applied directly to the original input cannot implement the XOR function. When $x_1 = 0$, the model's output must increase as x_2 increases. When $x_1 = 1$, the model's output must decrease as x_2 increases. A linear model must apply a fixed coefficient w_2 to x_2 . The linear model therefore cannot use the value of x_1 to change the coefficient on x_2 and cannot solve this problem. *(Right)* In the transformed space represented by the features extracted by a neural network, a linear model can now solve the problem. In our example solution, the two points that must have output 1 have been collapsed into a single point in feature space. In other words, the nonlinear features have mapped both $\mathbf{x} = [1, 0]^\top$ and $\mathbf{x} = [0, 1]^\top$ to a single point in feature space, $\mathbf{h} = [1, 0]^\top$. The linear model can now describe the function as increasing in h_1 and decreasing in h_2 . In this example, the motivation for learning the feature space is only to make the model capacity greater so that it can fit the training set. In more realistic applications, learned representations can also help the model to generalize.

→ In modern Neural networks, ~~and~~ it is recommended to use the rectified linear unit or ReLU, defined by the activation fn.

$$g(z) = \max\{0, z\}$$

i.e $f(x; W, c, w, b) = w^T \max\{0, w^T x + c\} + b$ — (1)

Now will solve XOR Problem

Let $W = \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix}$

$c = \begin{bmatrix} 0 \\ -1 \end{bmatrix}$, $w = \begin{bmatrix} 1 \\ -2 \end{bmatrix}$ & $b = 0$

→ Let X be i/p's matrix

i.e $X = \begin{bmatrix} 0 & 0 \\ 0 & 0 \\ 1 & 0 \\ 1 & 1 \end{bmatrix}$

first

$XW = \begin{bmatrix} 0 & 0 \\ 0 & 0 \\ 1 & 1 \\ 2 & 2 \end{bmatrix}$

Next, add bias vector c

i.e $\begin{bmatrix} 0 & -1 \\ 1 & 0 \\ 1 & 0 \\ 2 & 1 \end{bmatrix}$

ie $\begin{bmatrix} 0+0 & 0+(-1) \\ 0+1 & 1+(-1) \\ 0+1 & 1+(-1) \\ 0+2 & 2+(-1) \end{bmatrix}$

In this space, all the examples lie along slope 1, As we move along this line, the o/p needs to begin at 0, then rise to 1, then drop back down to 0.

A linear model cannot implement such a fn.

→ To compute 'h' here we need to apply the rectified linear Transformation.

$$\text{i.e. } \max(0, w^T x + c) + b$$

$$f(x; w, c, w, b) = \begin{bmatrix} 0 & 0 \\ 1 & 0 \\ 1 & 0 \\ 2 & 1 \end{bmatrix} \begin{matrix} \xrightarrow{\max(0,0) (0,-1)} = 0, 0 \\ \xrightarrow{\max(0,1) (0,0)} = 1, 0 \\ \vdots \end{matrix}$$

Now these points lie in a space where a linear model can solve the problem

→ Now multiply this by weight vector $w = \begin{bmatrix} 1 \\ -2 \end{bmatrix}$

i.e. $\begin{bmatrix} 0 \\ 1 \\ 1 \\ 0 \end{bmatrix}$

$$\begin{aligned} 0 \times 1 + 0 \times -2 &= 0 \\ 1 \times 1 + 0 \times -2 &= 1 \\ 1 \times 1 + 0 \times -2 &= 1 \\ 2 \times 1 + 1 \times -2 &= 0 \end{aligned}$$

which (solves) gives the XOR output for every example in the training data.

==

UNIT III –Part I

Deep Learning

Deep Feedforward Networks

Gradient Based Learning

Training a neural network is similar to training other machine learning models using **gradient descent**. The key difference between neural networks and simpler models (like linear regression) is that neural networks involve **nonlinearity**, making the loss function **non-convex**. This means:

- **No Guarantee of a Global Minimum** – Unlike linear regression, which can be solved using exact mathematical formulas, neural networks require **iterative optimization** methods like **Stochastic Gradient Descent (SGD)**.
- **Initialization Matters** – The starting values of the parameters (weights and biases) can significantly impact training. Weights are typically initialized to **small random values**, while biases are set to **zero or small positive values**.

Contd...

- Linear models (like logistic regression or support vector machines) can also be trained with gradient descent, but they have **global convergence guarantees**.
- In contrast, **stochastic gradient descent** (SGD) used for neural networks **does not guarantee convergence** and can get stuck in local minima or saddle points.
- Despite these challenges, gradient-based learning is highly effective for deep learning models.

Loss Function:

- Measures the error between the predicted output and the actual output for a single training example.
- Provides a score indicating how "wrong" a model is for a particular input.
- Examples: Mean Squared Error (MSE), Mean Absolute Error (MAE), Hinge Loss.
- Think of it as a "local" error measure for a single data point.

Cost Function:

- Calculates the average loss across the entire training dataset (or a mini-batch during training).
- Represents the overall performance of the model on the training data.
- It is calculated by averaging the loss values from all the training samples.
- Think of it as a "global" error measure for the entire dataset.
- It is usually minimized during model training to optimize performance.

Cost Functions in Neural Networks

A **cost function** measures how well a neural network is performing. Choosing the right cost function is crucial because it guides the optimization process.

1. Maximum Likelihood and Cross-Entropy Loss

- Most modern neural networks are trained using **maximum likelihood estimation (MLE)**. This means:
- We define a probability distribution $p(\mathbf{y} \mid \mathbf{x}; \boldsymbol{\theta})$ for the output.
- We optimize the parameters by maximizing the likelihood of the correct outputs.
- This results in the **negative log-likelihood (NLL)**, also called **cross-entropy loss**.

- **Cross-entropy loss** also known as log loss is a metric used in **machine learning** to measure the performance of a classification model. Its value ranges from 0 to 1 with lower being better. An ideal value would be 0. The goal of an optimizer tasked with training a classification model with cross-entropy loss would be to get the model as close to 0 as possible. In this article, we will delve into binary and multiclass cross-entropy losses and how to interpret the cross-entropy loss function.
- **In machine learning for classification tasks**, the model predicts the probability of a sample belonging to a particular class. Since each sample can belong to only a particular class, the true probability value would be 1 for that particular class and 0 for the other class(es). Cross entropy measures the difference between the predicted probability and the true probability.
- The Cross-Entropy Loss is derived from the principles of maximum likelihood estimation when applied to the task of classification. Maximizing the likelihood is equivalent to minimizing the negative log-likelihood. In classification, the likelihood function can be expressed as the product of the probabilities of the correct classes:
- **Binary Cross-Entropy Loss and Multiclass Cross-Entropy Loss** are two variants of cross-entropy loss, each tailored to different types of classification tasks. Let us see them in detail.

Binary Cross Entropy Loss

- Binary Cross-Entropy Loss is a widely used loss function in binary classification problems. For a dataset with N instances, the Binary Cross-Entropy Loss is calculated as:

- Where
$$-\frac{1}{N} \sum_{i=1}^N (y_i \cdot \log(p_i) + (1 - y_i) \log(1 - p_i))$$
 - y_i - true label for instance i
 - p_i - predicted probability, for instance, i by the model

Multiclass Cross Entropy Loss

- Multiclass Cross-Entropy Loss, also known as categorical cross-entropy or softmax loss, is a widely used loss function for training models in multiclass classification problems. For a dataset with N instances, Multiclass Cross-Entropy Loss is calculated as

- where
$$-\frac{1}{N} \sum_{i=1}^N \sum_{j=1}^C (y_{i,j} \cdot \log(p_{i,j}))$$
 - C is the number of classes.
 - $y_{i,j}$ are the true labels for class j for instance i
 - $p_{i,j}$ is the predicted probability for class j for instance i

2. Mean Square Error (MSE) / L2 Loss

- The Mean Square Error(MSE) or L2 loss is a loss function that quantifies the magnitude of the error between a machine learning algorithm prediction and an actual output by taking the average of the squared difference between the predictions and the target values. Squaring the difference between the predictions and actual target values results in a higher penalty assigned to more significant deviations from the target value. A mean of the errors normalizes the total errors against the number of samples in a dataset or observation.
- The mathematical equation for Mean Square Error (MSE) or L2 Loss is:

$$\text{MSE} = (1/n) * \sum (y_i - \bar{y})^2$$

- Where:

n is the number of samples in the dataset

y_i is the predicted value for the i-th sample

$\bar{y}$ is the target value for the i-th sample

- For **regression problems**, where the goal is to predict a continuous value, a common cost function is **Mean Squared Error (MSE)**
- This is equivalent to assuming that the output follows a normal (Gaussian) distribution.

3. Mean Absolute Error (MAE)

Mean Absolute Error (MAE), also known as L1 Loss, is a loss function used in regression tasks that calculates the average absolute differences between predicted values from a machine learning model and the actual target values. Unlike Mean Squared Error (MSE), MAE does not square the differences, treating all errors with equal weight regardless of their magnitude.

- The mathematical equation for Mean Absolute Error (MAE) or L1 Loss is:

$$\text{MAE} = (1/n) * \sum |y_i - \bar{y}|$$

Where:

n is the number of samples in the dataset

y_i is the predicted value for the i -th sample

$\bar{y}$ is the target value for the i -th sample

- Another option for regression is **Mean Absolute Error (MAE)**
- MSE finds the **mean** (average) of the target variable, while MAE finds the **median**.
- However, both MSE and MAE can cause issues with **gradient-based optimization** due to small gradients in certain cases.

Why Cross-Entropy is Preferred?

- Many activation functions (like sigmoid and softmax) involve exponentials, which can **saturate** (become very flat).
- This leads to very small gradients, making training slow.
- **Cross-entropy loss** works better because it includes a **log function**, which counteracts the exponential function, preventing saturation.

Summary

- **Neural networks are trained using gradient descent**, but unlike convex optimization, they don't guarantee finding the best solution.
- **Choosing the right cost function** is crucial:
 - **Cross-entropy loss** is used for classification.
 - **MSE or MAE** is used for regression.
- **Gradient computation is more complex** in neural networks but can be efficiently handled using **backpropagation**.

Output Units

The **output units** of a neural network determine how the final predictions are made. The choice of output units is closely tied to the **cost function** used for training. Different types of tasks (like regression or classification) require different types of output units. Let's go through the main types:

1. **Linear Units for Gaussian Output Distributions**
2. **Sigmoid Units for Bernoulli Output Distributions**
3. **Softmax Units for Multinoulli (Categorical) Output Distributions**
4. **Mixture Density Networks (MDNs)**
5. **Ordinal Regression Outputs**

Linear Units for Gaussian Output Distributions

- Used when the output follows a normal (Gaussian) distribution, which is common in regression tasks.
- A linear unit applies an affine transformation: $\hat{y} = \mathbf{W}^\top \mathbf{h} + \mathbf{b}$, where $\mathbf{h}$ represents the hidden features, $\mathbf{W}$ is the weight matrix, and $\mathbf{b}$ is the bias term.
- The output is a continuous value.
- The cost function used in this case is Mean Squared Error (MSE), which is equivalent to maximizing the likelihood of a Gaussian distribution.
- Advantage: No saturation problem, meaning the gradient remains useful during training.
- Challenge: Cannot easily represent variance changes unless additional mechanisms are introduced.

Sigmoid Units for Bernoulli Output Distributions

- Used when predicting binary outcomes (0 or 1).
- The output must be a probability between 0 and 1, which is achieved using the sigmoid activation function:
$$\hat{y} = \sigma(\mathbf{w}^\top \mathbf{h} + b)$$
- where σ is the logistic sigmoid function
- The cost function used is cross-entropy loss, which works well with probabilities.
- A linear unit might produce values outside the range $[0,1]$, requiring clamping, which leads to zero gradients and stops learning.
- Advantage: Ensures output is a valid probability.
- Challenge: Saturation problem—when the sigmoid output is close to 0 or 1, gradients become very small, slowing down learning.

Softmax Units for Multinoulli (Categorical) Output Distributions

- **Softmax** is a function used to convert a set of numbers (often called "logits") into probabilities. These probabilities represent a distribution over multiple possible outcomes (or classes).
- It generalizes the **sigmoid function** (used for binary classification) to handle cases with more than two classes.
- **Softmax is used:**

As an Output Layer in Classifiers:

- Softmax is commonly used in neural networks for classification tasks where the output is a probability distribution over multiple classes (e.g., predicting whether an image is a cat, dog, or bird).

Inside a Model:

- Sometimes, softmax is used within the model itself to make decisions between multiple internal options.

Contd...

1. **Input:** A vector of numbers $z = [z_1, z_2, \dots, z_n]$, where each z_i is an unnormalized log probability (often the output of a linear layer in a neural network).
2. **Output:** A vector of probabilities $\hat{y} = [\hat{y}_1, \hat{y}_2, \dots, \hat{y}_n]$, where:

$$\hat{y}_i = \text{softmax}(z)_i = \frac{\exp(z_i)}{\sum_{j=1}^n \exp(z_j)}$$

- This ensures that:
 - Each $\hat{y}_i$ is between 0 and 1.
 - The sum of all $\hat{y}_i$ is 1 (a valid probability distribution).

The exponential function ($\exp(\cdot)$) is used to ensure that:

All outputs are positive.

The log-likelihood (used in training) is well-behaved for gradient-based optimization.

The log-likelihood encourages the model to:

Increase the score z_i for the correct class.

Decrease the scores for incorrect classes.

Main properties of Softmax function are:

1. Invariance to Shifts:

- Adding a constant c to all inputs z_i doesn't change the output:

$$\text{softmax}(z) = \text{softmax}(z + c)$$

- This property is useful for numerical stability (e.g., subtracting the maximum value from z to avoid large exponentials).

2. Saturation:

- If one z_i is much larger than the others, its corresponding probability $\hat{y}_i$ will be close to 1, and the others will be close to 0. This is called "saturation."
- Saturation can cause issues during training if the loss function isn't designed to handle it.

So, Sigmoid: Used for binary classification (two classes). Outputs a single probability.

Softmax: Used for multi-class classification (n classes). Outputs a probability distribution over n classes.

To Train with Softmax the main objective is to

Maximize the log-likelihood of the correct class.

Behavior:

The model learns to predict probabilities that match the observed frequencies of classes in the training data.

If the model has enough capacity, it will approximate the true distribution.

Other Output Types

- Neural networks can have other types of output units based on the type of **prediction task**.
- Some examples:
 - **Mixture Density Networks (MDNs)**: Instead of a single value, they predict **multiple possible values** with different probabilities.
 - **Ordinal Regression Outputs**: Used when output values have an order (e.g., rating 1 to 5) but are not strictly continuous.
- Finally
 - ✓ **Linear units** are used for continuous outputs (regression).
 - ✓ **Sigmoid units** are used for binary classification.
 - ✓ **Softmax units** are used for multi-class classification.

The choice of output unit is linked to the **cost function**, which determines how well the network learns

Hidden Units

- Hidden units are the building blocks of neural networks. They are neurons in the hidden layers of a neural network. They transform input data through weights and activation functions, enabling the network to learn complex patterns. They take inputs, apply a transformation, and produce outputs that are passed to the next layer.
- Hidden units apply **non-linear transformations** to the input data. This allows the network to learn complex relationships between inputs and outputs.
- Without hidden units, a neural network would only be able to learn linear relationships, which are often insufficient for real-world tasks.
- Each hidden unit applies an **activation function** to its input. The choice of hidden units (activation functions) can significantly impact the performance of a neural network.
- **Common Types of Hidden Units**

Rectified Linear Units (ReLU)

Sigmoid and Hyperbolic Tangent (Tanh)

Other Hidden Unit Types

Properties of Good Activation Functions

- **Non-linearity:** The activation function must be non-linear to allow the network to learn complex patterns.
- **Differentiability:** The activation function should be differentiable (or at least have a subgradient) so that gradient-based optimization methods (like gradient descent) can be used to train the network.
- **Sparsity:** Some activation functions (like ReLU) can produce sparse outputs, which can make the network more efficient.
- The choice of activation function and the number of hidden units can significantly impact the performance of the network.

Rectified Linear Units(ReLU)

- **ReLU** is the most popular hidden unit. It's defined as:

$$g(z) = \max(0, z)$$

ReLU is the most commonly used activation function because it is computationally efficient and avoids the **vanishing gradient problem** (where gradients become very small, slowing down learning).

- If z is positive, output z .
- If z is negative, output 0.
- **Why ReLUs are great:**
 - They are simple and computationally efficient.
 - They avoid the **vanishing gradient problem** (gradients remain large when the unit is active).
 - They are close to linear, making optimization easier.

Drawbacks of ReLUs

- **Dead ReLUs:** If a ReLU outputs 0 for all inputs (e.g., due to bad initialization), it stops learning because the gradient is 0.
- **Solutions:**
 - Use **Leaky ReLU**: Allows a small gradient when $z < 0$:

$$g(z) = \max(0.01z, z)$$

- Use **Parametric ReLU (PReLU)**: Makes the slope for $z < 0$ a learnable parameter.
- Use **Maxout**: Divides inputs into groups and outputs the maximum value in each group. This can learn more complex functions.

Sigmoid & Hyperbolic Tangent (tanh)

Sigmoid Activation Function:

$$g(z) = \frac{1}{1 + e^{-z}}$$

tanh Activation Function:

$$g(z) = \tanh(z)$$

- Sigmoid squashes input values to a range between 0 and 1.
- Tanh squashes input values to a range between -1 and 1.
- These are more suitable for probabilistic models and are useful in recurrent networks.
- However, they saturate at extreme values leading to vanishing gradients.

Other Hidden Units

- **Radial Basis Function (RBF):** Used in certain specialized models.
- **Softmax Hidden Units:** Used in the output layer for classification tasks to produce probabilities for making categorical decisions.
- **Linear Units(Maxout):** A unit that generalizes ReLU and can learn piecewise linear functions. Used in some layers for dimensionality reduction.
- **Leaky ReLU:** A variant of ReLU that allows a small, non-zero gradient when the unit is not active. This helps prevent "dead" neurons that stop learning.
- **Parametric ReLU:** Similar to Leaky ReLU, but the slope for negative inputs is learned during training.

Challenges with Hidden Units

- **Vanishing Gradients:** In deep networks, gradients can become very small during backpropagation, making it hard for the network to learn. ReLU and its variants help mitigate this issue.
- **Dead Neurons:** In ReLU, some neurons can become "dead" (always outputting zero) if they are not activated during training. Leaky ReLU and Parametric ReLU address this problem.
- **Overfitting:** Too many hidden units can lead to overfitting, where the network memorizes the training data instead of generalizing to new data. Regularization techniques (like dropout) can help prevent this.

Architectural Design

- Architectural design refers to determining:

Number of layers (depth)

Number of neurons per layer (width)

How layers are connected

- The goal is to design a network that can effectively learn the task at hand, whether it's recognizing images, understanding text, or predicting outcomes.
- Most networks follow a **chain-like structure**, where each layer processes inputs from the previous layer. The mathematical representation is:
 - where:
$$h^{(1)} = g^{(1)}(W^{(1)}x + b^{(1)})$$
$$h^{(2)} = g^{(2)}(W^{(2)}h^{(1)} + b^{(2)})$$
 - h represents the layer outputs
 - w and b are weights and biases
 - g is the activation function

Key Components of Architecture Design

- **Depth of the Network:** This refers to the number of layers in the network. Deeper networks (more layers) can model more complex functions, but they are also harder to train and require more data.
- **Width of the Network:** This refers to the number of units (neurons) in each layer. Wider networks can capture more features, but they also require more computational resources.
- **Activation Functions:** These are functions applied to the output of each neuron to introduce non-linearity into the network. Common choices include ReLU (Rectified Linear Unit), sigmoid, and tanh.
- **Connections Between Layers:** In a standard feedforward network, each layer is fully connected to the next. However, other architectures (like convolutional networks) have different connection patterns.

Universal Approximation Theorem & Depth

- A **single hidden layer** can approximate any function, but may require too many neurons.
- **Deeper networks** (more layers) achieve better results with **fewer parameters** and often generalize better.
- **Task Complexity:** The complexity of the task (e.g., image recognition vs. simple classification) influences the architecture. More complex tasks may require deeper or wider networks.
- **Data Availability:** The amount of data available for training affects the architecture. With more data, you can train larger networks without overfitting.
- **Computational Resources:** The hardware available (e.g., GPUs, memory) can limit the size of the network you can train.
- **Theoretical result:** Any function can be approximated by a deep network with the right activation function (like sigmoid or ReLU).
- Depth improves learning efficiency but makes optimization harder

Other Considerations in Network Architecture

- While most networks follow a **simple chain structure**, many variations exist:
 - ☑ **Skip connections:** Bypass layers (e.g., ResNet) to improve gradient flow.
 - ☑ **Sparse connections:** Instead of full connectivity, only selected neurons interact (e.g., CNNs).
 - ☑ **Specialized architectures:**
- **Fully Connected Networks:** Every neuron in one layer is connected to every neuron in the next layer. This is the simplest form of a neural network.
- **Convolutional Networks (CNNs):** These are specialized for processing grid-like data. They use convolutional layers to capture spatial hierarchies in the data. **CNNs** for image processing
- **Recurrent Networks (RNNs):** These are designed for sequential data (e.g., time series, text). They have loops that allow information to persist over time. **RNNs** for sequence modeling
- **Transformers** for attention-based tasks.
- Deep networks show better generalization in real-world tasks. Simply increasing the number of parameters without increasing depth does not yield better results.
- However, the Key challenge is to find the right balance between **complexity & efficiency** through experimentation.

Back-Propagation and Other Differentiation Algorithms

- **Backpropagation** (short for "backward propagation of errors") is the algorithm used to train neural networks. It calculates the **gradient** of the loss function (a measure of how well the network is performing) with respect to each weight in the network.
 - The gradient tells us how much each weight contributes to the error, and we use this information to update the weights to reduce the error.
1. **Forward Propagation:** Inputs pass through layers, and the final output is computed.
 2. **Cost Calculation:** The difference between predicted and actual outputs is measured using a loss function.
 3. **Backward Propagation:** The error is sent back through the network to compute gradients, which guide parameter updates.

Mathematically:

$$\text{Gradient} = \frac{\partial J}{\partial W}$$

where J is the cost function and W represents weights.

4. Update Weights: The weights are updated using the gradients, typically with an optimization algorithm like **gradient descent**.

Visualizing the Process:

- A **computational graph** represents calculations as nodes and edges, helping in:
 - ✓ Breaking down complex functions into small operations.
 - ✓ Efficiently applying the **chain rule** for differentiation.
- The chain rule is the key mathematical tool behind back-propagation.

Backpropagation relies on the chain rule to compute gradients. The chain rule allows us to calculate the derivative of a composite function by multiplying the derivatives of its individual components.

For example:

If $z = f(g(x))$, then

$$\frac{dz}{dx} = \frac{dz}{dy} \times \frac{dy}{dx}$$

where $y=g(x)$

For a function with multiple layers:

$$h_3 = f_3(f_2(f_1(x)))$$

The gradient follows:

$$\frac{dh_3}{dx} = \frac{dh_3}{dh_2} \times \frac{dh_2}{dh_1} \times \frac{dh_1}{dx}$$

This allows gradients to flow backward layer by layer.

Gradient: The gradient is a vector of partial derivatives that points in the direction of the steepest increase in the loss function. By moving in the opposite direction (negative gradient), we reduce the loss.

General Steps of Back-Propagation

1. **Compute the forward pass** (calculate activations and loss).
2. **Initialize gradients** at the output layer.
3. **Apply chain rule** to compute gradients layer-by-layer.
4. **Update weights** using an optimization algorithm (e.g., **SGD**, **Adam**).

Challenges in Back-Propagation

- **Vanishing Gradients:** In deep networks, gradients shrink as they move backward, slowing learning. This makes it hard for the network to learn, especially in early layers.

Solution: Use **ReLU** activations.

- **Exploding Gradients:** Large gradients can destabilize training, gradients can become very large, causing the weights to update too drastically and destabilizing the training process.

Solution: Apply **gradient clipping**.

- **Computational Cost:** Large networks require efficient implementations like **mini-batch processing**.
- Techniques like proper weight initialization and activation functions help address them

Alternative Differentiation Algorithms

While backpropagation is the most widely used algorithm for computing gradients, there are other methods:

- **Symbolic Differentiation:** This involves deriving the exact mathematical expression for the gradient. It is precise but can be computationally expensive for large networks. Uses algebraic rules but may be inefficient.
- **Automatic Differentiation:** This is the method used in backpropagation. It combines the efficiency of symbolic differentiation with the flexibility of numerical differentiation. Used in frameworks like TensorFlow and PyTorch.
- **Forward and Reverse Mode Differentiation:**
 - Forward Mode:** Efficient for small networks.
 - Reverse Mode (Back-propagation):** Best for deep learning
- **Numerical Differentiation:** This approximates gradients by computing small changes in the loss function. It is simple but slow and prone to numerical errors.

UNIT III –Part II

Deep Learning

Regularization for Deep Learning

Regularization

- **Regularization** is a technique used to prevent a model from overfitting the training data. Overfitting occurs when a model learns the training data too well, including its noise and outliers, and performs poorly on new, unseen data.
- Regularization techniques add constraints or penalties to the model to make it simpler and more generalizable.
- This penalty discourages the model from becoming too complex or having large parameter values, which helps in controlling the model's ability to fit noise in the training data

Addressing generalization issues

- To fix underfitting, deepen your neural network by adding more layers.
- To fix overfitting, reduce the model's capacity by removing layers and thus reducing the number of parameters.
- Another way to reduce capacity is to limit the growth of the weights through some kind of weight decay process. By decaying weights, in effect you're limiting the range of potential networks to choose from.
- The best way to help a model generalize is to gather a larger data set. But this is not always possible. If you don't have access to more data, you can use regularization methods.
- The aim to close the performance gap between the test set and training set while keeping training performance as high as possible.
- Regularization will help you do so.

Regularization techniques

- L1 Regularization
- L2 regularization
- Early stopping
- Data augmentation
- dropout.

Parameter Norm Penalties

- **Parameter norm penalties** are a type of regularization that adds a penalty to the loss function based on the size of the model's parameters (weights).
- By penalizing large weights, the model is encouraged to learn simpler patterns that generalize better to new data.
- ◆ The regularized objective function looks like:

$$J_{\text{reg}}(\theta) = J(\theta) + \alpha\Omega(\theta)$$

where:

- $J(\theta)$ is the original loss function,
- $\Omega(\theta)$ is the norm penalty,
- α is a hyperparameter controlling the strength of regularization

Types of Parameter Norm Penalties

There are two main types of parameter norm penalties:

L2 Regularization (Weight Decay)

L1 Regularization

L2 Regularization - Ridge regression (Weight Decay)

- Adds a penalty proportional to the **square of the weights** (L2 norm).
- Encourages **small weights** but does not drive them to zero
- Helps **smooth predictions** and prevent extreme parameter values.
- The loss function becomes:

$$\text{Loss} = \text{Original Loss} + \lambda \sum_i w_i^2$$

- where λ is a hyperparameter that controls the strength of the penalty, and w_i are the weights.
- L2 regularization encourages the weights to be small but not exactly zero, leading to smoother models.
- Commonly used in Deep networks, Logistic Regression, Linear regression

L1 Regularization- Lasso Regularization

- Adds a penalty proportional to the **absolute value of the weights** (L1 norm).
- Encourages **sparse weights** (some weights become exactly zero).
- Helps with **feature selection** by automatically ignoring unimportant features
- The loss function becomes:

$$\text{Loss} = \text{Original Loss} + \lambda \sum_i |w_i|$$

- L1 regularization encourages some weights to become exactly zero, leading to **sparse models** (models with fewer active weights).
- Commonly used in Compression sensing, feature selection models.

Aspect	L1 Regularization	L2 Regularization
Effect on Weights	Encourages sparsity (some weights = 0)	Shrinks weights smoothly
Feature Selection	Selects important features automatically	Keeps all features but reduces importance
Used In	Sparse models, feature selection	Most deep learning models

Uses of Parameter Norm Penalties:

- **Prevent Overfitting:** By penalizing large weights, the model is less likely to fit the noise in the training data and more likely to generalize to new data.
- **Control Model Complexity:** Regularization helps control the complexity of the model, making it simpler and easier to interpret.
- **Improve Optimization:** Regularization can make the optimization process more stable by preventing the weights from growing too large.
- Parameter norm penalties like L1 and L2 regularization help prevent overfitting and improve generalization. The choice depends on the problem.
- Use L2 regularization (weight decay) for smooth learning.
- Use L1 regularization when feature selection or sparsity is needed.
- Fine-tune the regularization strength for the best performance
- λ is a hyperparameter that controls the strength of the regularization.
- A small λ means less regularization, while a large λ means stronger regularization.
- The optimal value of λ is typically found through cross-validation or experimentation.

Under-Constrained Problems

- Some machine learning problems have **more parameters than data points**, leading to **multiple solutions** (underdetermined).
 - ◇ Regularization helps stabilize solutions by controlling parameter magnitudes.
 - ◇ Example: **Logistic regression on linearly separable data** may cause **infinite weight growth**, leading to instability.
 - ◇ Solution: **Weight decay (L2 regularization)** ensures convergence by penalizing large weights

Dataset Augmentation

- **Artificially increasing dataset size** using transformations on existing data like:
 - ☑ **Image:** Rotation, flipping, cropping, color jittering, scaling, shifting.
 - ☑ **Text:** Synonym replacement, back-translation.
 - ☑ **Audio:** Speed changes, background noise.
- Helps deep models generalize better, especially for **image classification** tasks like **object recognition**.
- Some transformations change meaning (e.g., flipping 'b' to 'd' in OCR).
- Introduces variability into the training data, making the model more robust to variations in the input.
- Reduces overfitting by providing more diverse examples for the model to learn from.

Noise Robustness

- **Adding noise to inputs or weights** can help the model generalize by preventing it from memorizing exact data patterns.

Methods include:

- **Gaussian noise** (random variations in input values).
- **Dropout** (randomly deactivating neurons).
- **Jittering** (slightly shifting inputs).

Helps prevent **overfitting** and improves **robustness to unseen data**

Semi-Supervised Learning

- **Uses both labeled and unlabeled data** to improve learning.
 - ◇ Works well when **labeling is expensive** but **unlabeled data is abundant**.

Techniques:

- **Self-training:** Model predicts labels for unlabeled data and retrains.
- **Consistency regularization:** Encourages consistent predictions for similar inputs

Multi-Task Learning

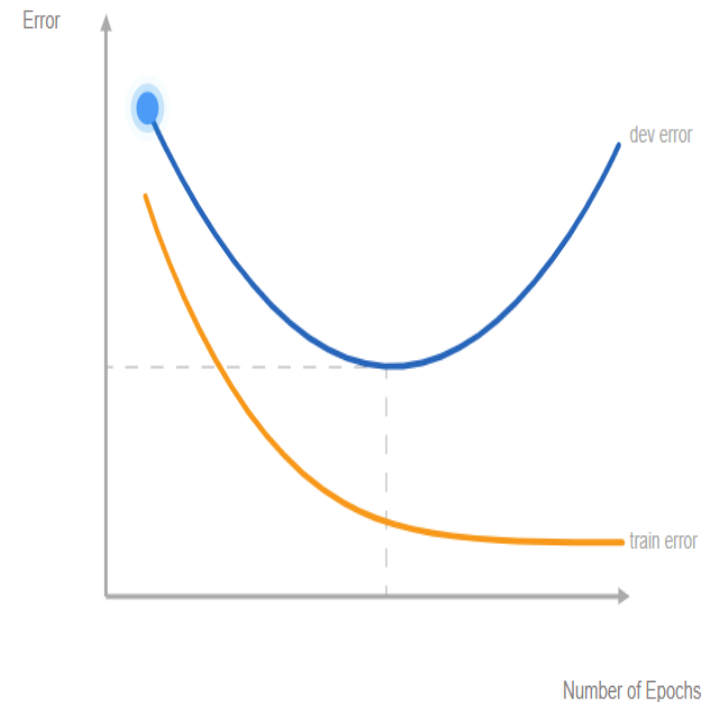
- **Multi-task learning (MTL)** is a machine learning paradigm where a model is trained to perform multiple related tasks simultaneously, sharing representations across them.
- **Learning multiple related tasks together** improves overall performance.
 - ◇ Helps the model **leverage shared features** and **reduce overfitting**.
- A single neural network is trained on multiple tasks (e.g., object detection + segmentation, sentiment analysis + topic classification).
- The model shares some layers (common feature extractor) while having task-specific heads (output layers).

Example:

- Training a model to recognize **faces, emotions, and age** from the same image

Early Stopping

- One way to stop a neural network from overfitting is to halt the training process before the network has thoroughly memorized the data set. This widely used technique is called early stopping.
- **Stopping training when validation loss stops improving** , prevents overfitting.
Requires **monitoring validation loss** during training.
Works like an **adaptive form of regularization** without modifying the loss function



Dropout

- **Randomly disables neurons during training** to prevent over-reliance on specific features.
- At every iteration, it randomly selects some nodes and removes them along with all of their incoming and outgoing connections.
- Each iteration has a different set of nodes, which results in a different set of outputs. Works like **ensemble learning** by forcing different subnetworks to learn independently.
- This probability of choosing how many nodes should be dropped is the hyperparameter of the dropout function.
- Helps in **reducing overfitting** and improving **generalization**.

